

목표

학습 목표

- 관련 함수를 묶어 클래스를 만들고, 객체들이 협력하여 하나의 큰 기능을 수행하도록 한다.
- 클래스와 함수에 대한 단위 테스트를 통해 의도한 대로 정확하게 작동하는 영역을 확보한다.
- [3주 차 공통 피드백](#)을 최대한 반영한다.
- [비공개 저장소 과제 진행 가이드](#)를 참고하여 새로운 방식으로 과제 제출물을 제출한다.

회고

아래 질문에 대한 최종 회고를 진행하고 소감에 구체적인 결과를 작성한다. 소감은 텍스트로 작성해야 하며 외부 링크는 허용하지 않는다.

- 지원서나 중간 회고에서 현실적인 목표를 설정하고 이를 달성했다고 생각하나요? 그 이유는 무엇인가요?
- 중간 회고에서 조정한 목표가 실제 목표 달성에 도움이 되었나요? 목표를 달성하는 데 어떤 점이 효과적이었다고 생각하나요?
- 각 미션의 목표를 달성하기 위해 세운 계획을 잘 이행했나요? 그 과정에서 어떤 전략이 효과가 있었나요?
- 몰입하고 함께 성장하는 과정을 통해 인상 깊었던 경험이나 변화가 있었나요?

프리코스 진행 방식

진행 방식

- 미션은 과제 진행 요구 사항, 기능 요구 사항, 프로그래밍 요구 사항 세 가지로 구성되어 있다.
- 세 개의 요구 사항을 만족하기 위해 노력한다. 특히 기능을 구현하기 전에 기능 목록을 만들고, 기능 단위로 커밋 하는 방식으로 진행한다.
- 기능 요구 사항에 기재되지 않은 내용은 스스로 판단하여 구현한다.
- 매주 진행할 미션은 화요일 오후 3시부터 확인할 수 있으며, 다음 주 월요일까지 구현을 완료하여 제출해야 한다. 제출은 일요일 오후 3시부터 가능하다.
 - 정해진 시간을 지키지 않을 경우 미션을 제출하지 않은 것으로 간주한다.
 - 정해진 시간 내에 우아한테크코스 계정을 협력자로 초대하지 않으면 제출하지 않은 것으로 간주한다.
 - 종료 일시 이후에는 추가 푸시를 허용하지 않는다.

미션 제출 방법

- 미션 구현을 완료한 후 GitHub을 통해 제출해야 한다.
 - GitHub을 활용한 제출 방법은 [프리코스 과제 제출](#) 문서를 참고해 제출한다.
- GitHub에 미션을 제출한 후 [우아한테크코스 지원 플랫폼](#)에서 과제 제출물을 최종 제출한다.

- 자세한 안내는 [제출 가이드](#)를 참고한다.
- 과제를 수행하면서 느낀 점, 배운 점, 많은 시간을 투자한 부분 등 자유롭게 작성한다.

과제 제출 전 체크 리스트

- 기능을 올바르게 구현했더라도 요구 사항에 명시된 출력 형식을 따르지 않으면 0점을 받게 된다.
- 기능 구현을 완료한 후 아래 가이드에 따라 모든 테스트가 성공적으로 실행되는지 확인한다.
- 테스트가 실패하면 점수가 0점이 되므로 제출하기 전에 반드시 확인한다.

테스트 실행 가이드

- 터미널에서 `java -version` 을 실행하여 Java 버전이 21인지 확인한다. Eclipse 또는 IntelliJ IDEA와 같은 IDE에서 Java 21로 실행되는지 확인한다.
- 터미널에서 Mac 또는 Linux 사용자의 경우 `./gradlew clean test` 명령을 실행하고, Windows 사용자의 경우 `gradlew.bat clean test` 또는 `.\gradlew.bat clean test` 명령을 실행할 때 모든 테스트가 아래와 같이 통과하는지 확인한다.

BUILD SUCCESSFUL in 0s

편의점

과제 진행 요구 사항

- 미션은 [편의점](#) 저장소를 생성하는 것으로 시작한다.
- 기능을 구현하기 전 `README.md` 에 구현할 기능 목록을 정리해 추가한다.
- Git의 커밋 단위는 앞 단계에서 `README.md` 에 정리한 기능 목록 단위로 추가한다.
 - [AngularJS Git Commit Message Conventions](#)을 참고해 커밋 메시지를 작성한다.
- 자세한 과제 진행 방법은 프리코스 진행 가이드 문서를 참고한다.

기능 요구 사항

구매자의 할인 혜택과 재고 상황을 고려하여 최종 결제 금액을 계산하고 안내하는 결제 시스템을 구현한다.

- 사용자가 입력한 상품의 가격과 수량을 기반으로 최종 결제 금액을 계산한다.
 - 총구매액은 상품별 가격과 수량을 곱하여 계산하며, 프로모션 및 멤버십 할인 정책을 반영하여 최종 결제 금액을 산출한다.
- 구매 내역과 산출한 금액 정보를 영수증으로 출력한다.
- 영수증 출력 후 추가 구매를 진행할지 또는 종료할지를 선택할 수 있다.

- 사용자가 잘못된 값을 입력할 경우 `IllegalArgumentException` 를 발생시키고, "[ERROR]"로 시작하는 에러 메시지를 출력 후 그 부분부터 입력을 다시 받는다.
 - `Exception` 이 아닌 `IllegalArgumentException` , `IllegalStateException` 등과 같은 명확한 유형을 처리한다.

재고 관리

- 각 상품의 재고 수량을 고려하여 결제 가능 여부를 확인한다.
- 고객이 상품을 구매할 때마다, 결제된 수량만큼 해당 상품의 재고에서 차감하여 수량을 관리한다.
- 재고를 차감함으로써 시스템은 최신 재고 상태를 유지하며, 다음 고객이 구매할 때 정확한 재고 정보를 제공한다.

프로모션 할인

- 오늘 날짜가 프로모션 기간 내에 포함된 경우에만 할인을 적용한다.
- 프로모션은 N개 구매 시 1개 무료 증정(Buy N Get 1 Free)의 형태로 진행된다.
- 1+1 또는 2+1 프로모션이 각각 지정된 상품에 적용되며, 동일 상품에 여러 프로모션이 적용되지 않는다.
- 프로모션 혜택은 프로모션 재고 내에서만 적용할 수 있다.
- 프로모션 기간 중이라면 프로모션 재고를 우선적으로 차감하며, 프로모션 재고가 부족할 경우에는 일반 재고를 사용한다.
- 프로모션 적용이 가능한 상품에 대해 고객이 해당 수량보다 적게 가져온 경우, 필요한 수량을 추가로 가져오면 혜택을 받을 수 있음을 안내한다.
- 프로모션 재고가 부족하여 일부 수량을 프로모션 혜택 없이 결제해야 하는 경우, 일부 수량에 대해 정가로 결제하게 됨을 안내한다.

멤버십 할인

- 멤버십 회원은 프로모션 미적용 금액의 30%를 할인받는다.
- 프로모션 적용 후 남은 금액에 대해 멤버십 할인을 적용한다.
- 멤버십 할인의 최대 한도는 8,000원이다.

영수증 출력

- 영수증은 고객의 구매 내역과 할인을 요약하여 출력한다.
- 영수증 항목은 아래와 같다.
 - 구매 상품 내역: 구매한 상품명, 수량, 가격
 - 증정 상품 내역: 프로모션에 따라 무료로 제공된 증정 상품의 목록
 - 금액 정보
 - 총구매액: 구매한 상품의 총 수량과 총 금액
 - 행사할인: 프로모션에 의해 할인된 금액
 - 멤버십할인: 멤버십에 의해 추가로 할인된 금액
 - 내실돈: 최종 결제 금액
- 영수증의 구성 요소를 보기 좋게 정렬하여 고객이 쉽게 금액과 수량을 확인할 수 있게 한다.

입출력 요구 사항

입력

- 구현에 필요한 상품 목록과 행사 목록을 파일 입출력을 통해 불러온다.
 - `src/main/resources/products.md` 과 `src/main/resources/promotions.md` 파일을 이용한다.
 - 두 파일 모두 내용의 형식을 유지한다면 값은 수정할 수 있다.
- 구매할 상품과 수량을 입력 받는다. 상품명, 수량은 하이픈(-)으로, 개별 상품은 대괄호([])로 묶어 쉼표(,)로 구분한다.

[콜라-10],[사이다-3]

- 프로모션 적용이 가능한 상품에 대해 고객이 해당 수량보다 적게 가져온 경우, 그 수량만큼 추가 여부를 입력받는다.
 - Y: 증정 받을 수 있는 상품을 추가한다.
 - N: 증정 받을 수 있는 상품을 추가하지 않는다.

Y

- 프로모션 재고가 부족하여 일부 수량을 프로모션 혜택 없이 결제해야 하는 경우, 일부 수량에 대해 정가로 결제할지 여부를 입력받는다.
 - Y: 일부 수량에 대해 정가로 결제한다.
 - N: 정가로 결제해야 하는 수량만큼 제외한 후 결제를 진행한다.

Y

- 멤버십 할인 적용 여부를 입력 받는다.
 - Y: 멤버십 할인을 적용한다.
 - N: 멤버십 할인을 적용하지 않는다.

Y

- 추가 구매 여부를 입력 받는다.
 - Y: 재고가 업데이트된 상품 목록을 확인 후 추가로 구매를 진행한다.
 - N: 구매를 종료한다.

Y

출력

- 환영 인사와 함께 상품명, 가격, 프로모션 이름, 재고를 안내한다. 만약 재고가 0개라면 재고 없음 을 출력한다.

안녕하세요. W편의점입니다.

현재 보유하고 있는 상품입니다.

- 콜라 1,000원 10개 탄산2+1
- 콜라 1,000원 10개
- 사이다 1,000원 8개 탄산2+1
- 사이다 1,000원 7개
- 오렌지주스 1,800원 9개 MD추천상품
- 오렌지주스 1,800원 재고 없음
- 탄산수 1,200원 5개 탄산2+1
- 탄산수 1,200원 재고 없음
- 물 500원 10개
- 비타민워터 1,500원 6개
- 감자칩 1,500원 5개 반짝할인
- 감자칩 1,500원 5개
- 초코바 1,200원 5개 MD추천상품
- 초코바 1,200원 5개
- 에너지바 2,000원 5개
- 정식도시락 6,400원 8개
- 컵라면 1,700원 1개 MD추천상품
- 컵라면 1,700원 10개

구매하실 상품명과 수량을 입력해 주세요. (예: [사이다-2], [감자칩-1])

- 프로모션 적용이 가능한 상품에 대해 고객이 해당 수량만큼 가져오지 않았을 경우, 혜택에 대한 안내 메시지를 출력한다.

현재 {상품명}은(는) 1개를 무료로 더 받을 수 있습니다. 추가하시겠습니까? (Y/N)

- 프로모션 재고가 부족하여 일부 수량을 프로모션 혜택 없이 결제해야 하는 경우, 일부 수량에 대해 정가로 결제할지 여부에 대한 안내 메시지를 출력한다.

현재 {상품명} {수량}개는 프로모션 할인이 적용되지 않습니다. 그래도 구매하시겠습니까? (Y/N)

- 멤버십 할인 적용 여부를 확인하기 위해 안내 문구를 출력한다.

멤버십 할인을 받으시겠습니까? (Y/N)

- 구매 상품 내역, 증정 상품 내역, 금액 정보를 출력한다.

```
=====W 편의점=====
상품명      수량      금액
콜라        3        3,000
에너지바    5        10,000
=====증정=====
콜라        1
=====
총구매액      8        13,000
행사할인              -1,000
멤버십할인              -3,000
내실돈              9,000
```

- 추가 구매 여부를 확인하기 위해 안내 문구를 출력한다.

감사합니다. 구매하고 싶은 다른 상품이 있나요? (Y/N)

- 사용자가 잘못된 값을 입력했을 때, "[ERROR]"로 시작하는 오류 메시지와 함께 상황에 맞는 안내를 출력한다.
 - 구매할 상품과 수량 형식이 올바르지 않은 경우: [ERROR] 올바르지 않은 형식으로 입력했습니다. 다시 입력해 주세요.
 - 존재하지 않는 상품을 입력한 경우: [ERROR] 존재하지 않는 상품입니다. 다시 입력해 주세요.
 - 구매 수량이 재고 수량을 초과한 경우: [ERROR] 재고 수량을 초과하여 구매할 수 없습니다. 다시 입력해 주세요.
 - 기타 잘못된 입력의 경우: [ERROR] 잘못된 입력입니다. 다시 입력해 주세요.

실행 결과 예시

안녕하세요. W편의점입니다.
현재 보유하고 있는 상품입니다.

- 콜라 1,000원 10개 탄산2+1
- 콜라 1,000원 10개
- 사이다 1,000원 8개 탄산2+1
- 사이다 1,000원 7개
- 오렌지주스 1,800원 9개 MD추천상품
- 오렌지주스 1,800원 재고 없음
- 탄산수 1,200원 5개 탄산2+1
- 탄산수 1,200원 재고 없음

- 물 500원 10개
- 비타민워터 1,500원 6개
- 감자칩 1,500원 5개 반짝할인
- 감자칩 1,500원 5개
- 초코바 1,200원 5개 MD추천상품
- 초코바 1,200원 5개
- 에너지바 2,000원 5개
- 정식도시락 6,400원 8개
- 컵라면 1,700원 1개 MD추천상품
- 컵라면 1,700원 10개

구매하실 상품명과 수량을 입력해 주세요. (예: [사이다-2], [감자칩-1])
[콜라-3], [에너지바-5]

멤버십 할인을 받으시겠습니까? (Y/N)

Y

=====W 편의점=====

상품명	수량	금액
콜라	3	3,000
에너지바	5	10,000
=====증	정=====	
콜라	1	

총구매액	8	13,000
행사할인		-1,000
멤버십할인		-3,000
내실돈		9,000

감사합니다. 구매하고 싶은 다른 상품이 있나요? (Y/N)

Y

안녕하세요. W편의점입니다.

현재 보유하고 있는 상품입니다.

- 콜라 1,000원 7개 탄산2+1
- 콜라 1,000원 10개
- 사이다 1,000원 8개 탄산2+1
- 사이다 1,000원 7개
- 오렌지주스 1,800원 9개 MD추천상품
- 오렌지주스 1,800원 재고 없음
- 탄산수 1,200원 5개 탄산2+1
- 탄산수 1,200원 재고 없음
- 물 500원 10개
- 비타민워터 1,500원 6개
- 감자칩 1,500원 5개 반짝할인
- 감자칩 1,500원 5개
- 초코바 1,200원 5개 MD추천상품
- 초코바 1,200원 5개
- 에너지바 2,000원 재고 없음
- 정식도시락 6,400원 8개
- 컵라면 1,700원 1개 MD추천상품

- 컵라면 1,700원 10개

구매하실 상품명과 수량을 입력해 주세요. (예: [사이다-2], [감자칩-1])
[콜라-10]

현재 콜라 4개는 프로모션 할인이 적용되지 않습니다. 그래도 구매하시겠습니까? (Y/N)
Y

멤버십 할인을 받으시겠습니까? (Y/N)
N

=====W 편의점=====

상품명	수량	금액
콜라	10	10,000

=====증정=====

콜라	2	
----	---	--

=====

총구매액	10	10,000
------	----	--------

행사할인		-2,000
------	--	--------

멤버십할인		-0
-------	--	----

내실돈		8,000
-----	--	-------

감사합니다. 구매하고 싶은 다른 상품이 있나요? (Y/N)
Y

안녕하세요. W편의점입니다.
현재 보유하고 있는 상품입니다.

- 콜라 1,000원 재고 없음 탄산2+1
- 콜라 1,000원 7개
- 사이다 1,000원 8개 탄산2+1
- 사이다 1,000원 7개
- 오렌지주스 1,800원 9개 MD추천상품
- 오렌지주스 1,800원 재고 없음
- 탄산수 1,200원 5개 탄산2+1
- 탄산수 1,200원 재고 없음
- 물 500원 10개
- 비타민워터 1,500원 6개
- 감자칩 1,500원 5개 반짝할인
- 감자칩 1,500원 5개
- 초코바 1,200원 5개 MD추천상품
- 초코바 1,200원 5개
- 에너지바 2,000원 재고 없음
- 정식도시락 6,400원 8개
- 컵라면 1,700원 1개 MD추천상품
- 컵라면 1,700원 10개

구매하실 상품명과 수량을 입력해 주세요. (예: [사이다-2], [감자칩-1])
[오렌지주스-1]

현재 오렌지주스는(는) 1개를 무료로 더 받을 수 있습니다. 추가하시겠습니까? (Y/N)
Y

멤버십 할인을 받으시겠습니까? (Y/N)

Y

=====W 편의점=====

상품명	수량	금액
오렌지주스	2	3,600

=====증정=====

오렌지주스	1	
-------	---	--

=====

총구매액	2	3,600
행사할인		-1,800
멤버십할인		-0
내실돈		1,800

감사합니다. 구매하고 싶은 다른 상품이 있나요? (Y/N)

N

프로그래밍 요구 사항 1

- JDK 21 버전에서 실행 가능해야 한다.
- 프로그램 실행의 시작점은 `Application` 의 `main()` 이다.
- `build.gradle` 파일은 변경할 수 없으며, 제공된 라이브러리 이외의 외부 라이브러리는 사용하지 않는다.
- 프로그램 종료 시 `System.exit()` 를 호출하지 않는다.
- 프로그래밍 요구 사항에서 달리 명시하지 않는 한 파일, 패키지 등의 이름을 바꾸거나 이동하지 않는다.
- 자바 코드 컨벤션을 지키면서 프로그래밍한다.
 - 기본적으로 [Java Style Guide](#)를 원칙으로 한다.

프로그래밍 요구 사항 2

- indent(인덴트, 들여쓰기) depth를 3이 넘지 않도록 구현한다. 2까지만 허용한다.
 - 예를 들어 while문 안에 if문이 있으면 들여쓰기는 2이다.
 - 힌트: indent(인덴트, 들여쓰기) depth를 줄이는 좋은 방법은 함수(또는 메서드)를 분리하면 된다.
- 3항 연산자를 쓰지 않는다.
- 함수(또는 메서드)가 한 가지 일만 하도록 최대한 작게 만들어라.
- JUnit 5와 AssertJ를 이용하여 정리한 기능 목록이 정상적으로 작동하는지 테스트 코드로 확인한다.
 - 테스트 도구 사용법이 익숙하지 않다면 아래 문서를 참고하여 학습한 후 테스트를 구현한다.
 - [JUnit 5 User Guide](#)
 - [AssertJ User Guide](#)
 - [AssertJ Exception Assertions](#)
 - [Guide to JUnit 5 Parameterized Tests](#)

프로그래밍 요구 사항 3

- else 예약어를 쓰지 않는다.
 - else를 쓰지 말라고 하니 switch/case로 구현하는 경우가 있는데 switch/case도 허용하지 않는다.
 - 힌트: if 조건절에서 값을 return하는 방식으로 구현하면 else를 사용하지 않아도 된다.
- Java Enum을 적용하여 프로그램을 구현한다.
- 구현한 기능에 대한 단위 테스트를 작성한다. 단, UI(System.out, System.in, Scanner) 로직은 제외한다.

프로그래밍 요구 사항 4

- 함수(또는 메서드)의 길이가 10라인을 넘어가지 않도록 구현한다.
 - 함수(또는 메서드)가 한 가지 일만 잘 하도록 구현한다.
- 입출력을 담당하는 클래스를 별도로 구현한다.
 - 아래 `InputView`, `OutputView` 클래스를 참고하여 입출력 클래스를 구현한다.
 - 클래스 이름, 메소드 반환 유형, 시그니처 등은 자유롭게 수정할 수 있다.

```
public class InputView {
    public String readItem() {
        System.out.println("구매하실 상품명과 수량을 입력해 주세요. (예: [사이다-2],[감자칩-1])");
        String input = Console.readLine();
        // ...
    }
    // ...
}
```

```
public class OutputView {
    public void printProducts() {
        System.out.println("- 콜라 1,000원 10개 탄산2+1");
        // ...
    }
    // ...
}
```

라이브러리

- `camp.nextstep.edu.missionutils` 에서 제공하는 `DateTimes` 및 `Console` API를 사용하여 구현해야 한다.
 - 현재 날짜와 시간을 가져오려면 `camp.nextstep.edu.missionutils.DateTimes` 의 `now()` 를 활용한다.
 - 사용자가 입력하는 값은 `camp.nextstep.edu.missionutils.Console` 의 `readLine()` 을 활용한다.